

doble y triple

```
Dame un numero del 1 al 100:
22
doble: 44
triple: 66

Process returned 0 (0x0)   execution time : 7.507
s
Press any key to continue.
```

```
7 void f_triple(int *a){
8     (*a) = (*a) * 3;
9 }
10
11 int main()
12 {
13     int a;
14     printf("Dame un numero del 1 al 100:\n");
15     scanf("%d", &a);
16
17     f_doble(a);
18     printf("doble: %d\n", f_doble(a));
19
20     f_triple(&a);
21     printf("triple: %d\n", a);
22     return 0;
23 }
```

NOTAS

Acceso equivalente
Estos dos expresiones acceden al mismo valor
número [22] * (número + 2)

permite acceder al valor almacenado en la dirección que apunta
punto, variable que almacena la dirección de memoria de otra variable

Esto ocurre porque un arreglo puede entenderse como un puntero al primer elemento más del desplazamiento.

outing = vob otuoc/A

Malloc

memory allocation

pide memoria al sistema

sizeof(int) obtiene tamaño real

devuelve una dirección de memoria

Asigna dinámicamente un

bloque de memoria en bytes

```
int x = 5;
```

```
int *v;
```

```
v = (int*) malloc(5 * sizeof(int));
```

```
free(v);
```

si no se usa free, ocurre fuga de memoria

Heap

Área de memoria
de donde asignamos
variables dinámicas

Recursividad

Si no tiene caso base, para **stack overflow**

Es cuando una función se llama a sí misma

Caso base indispensable para evitar ejecución infinita

Toda función recursiva necesita una condición para detenerse

Memoria Dinámica

Es memoria reservada durante la ejecución del programa.

Permite crear arreglos o estructuras cuyo tamaño se decide cuando el programa ya está corriendo

Calloc y Malloc

Calloc inicializa

a la memoria asignada a cero, y reserva

Malloc no lo hace, reserva memoria, obtiene tamaño real de

entero $\text{sizeof}(\text{int})$ y devuelve la dirección de memoria

Realloc

Permite cambiar tamaño de memoria ya reservada.

```
v = (int*) realloc(v, 10 * sizeof(int));
```

Se le da a 10 enteros

Variables automáticas y variable dinámicas
Las automáticas se gestionan en el stack (pila) y las dinámicas en el heap

Variables automáticas

Int, float, char
pequeño e ilimitado

Variables dinámicas

malloc() o calloc()
grande y flexible

Se necesitan 3 bucles anidados para implementar la lógica estandar de multiplicación de matrices

Primer bucle (Externo i)

recorre las filas de la primera matriz. Su función principal es determinar en qué fila de la matriz resultante se va almacenar el valor final que se está calculando

Segundo bucle (Intermedio j)

Recorre las columnas de la segunda matriz. Al combinarse con el índice $C[i][j]$, donde se guardará la suma acumulada de las multiplicaciones

Tercer bucle (Interno - k)

Se encarga de realizar la multiplicación y suma consecutiva

• Malloc, ¿Por qué es importante `sizeof(tipo)` en malloc por un arreglo?

Malloc no entiende los tipos de datos (enteros, caracteres) sino únicamente bytes `sizeof(tipo)`, el código funciona correctamente sin necesidad de cambiar números

Puntero

```
int x = 5
```

```
int *p = &x
```

```
printf("%d\n", *p);
```

Apunta desde esta
↓ la memoria

Imprime 5

Cuando defines un arreglo el identificador es un puntero.

* En una matriz bidimensional, la fórmula para calcular la dirección del elemento $v[i][j]$

$\text{Base} + \underbrace{C_i \times \text{columna} + j}_{\text{elementos debe saltar}} \times \underbrace{\text{sizeof}(A_{ip})}_{\text{para convertirlos a bytes reales}}$

→ dirección de memoria del primer elemento